Chapter 3: Kernel

Introduction and Architecture of Kernel

→ A Kernel is the core component of an operating system. Using interprocess communication and system calls, it acts as a bridge between applications and the data processing performed at the hardware level.

→ The Kernel is responsible for low-level tasks such as disk management, task management and memory management.

→ A Computer Kernel interfaces between the three major computer hardware components, providing services betⁿ the application / user interface and the CPU, memory and other hardware I/O devices.

→ The Kernel is responsible for
i) process management for application execution
ii) Memory management, allocation and I/O
iii) Device management through the use of device drivers.
iv) System call control, which is essential for the execution of Kernel services.

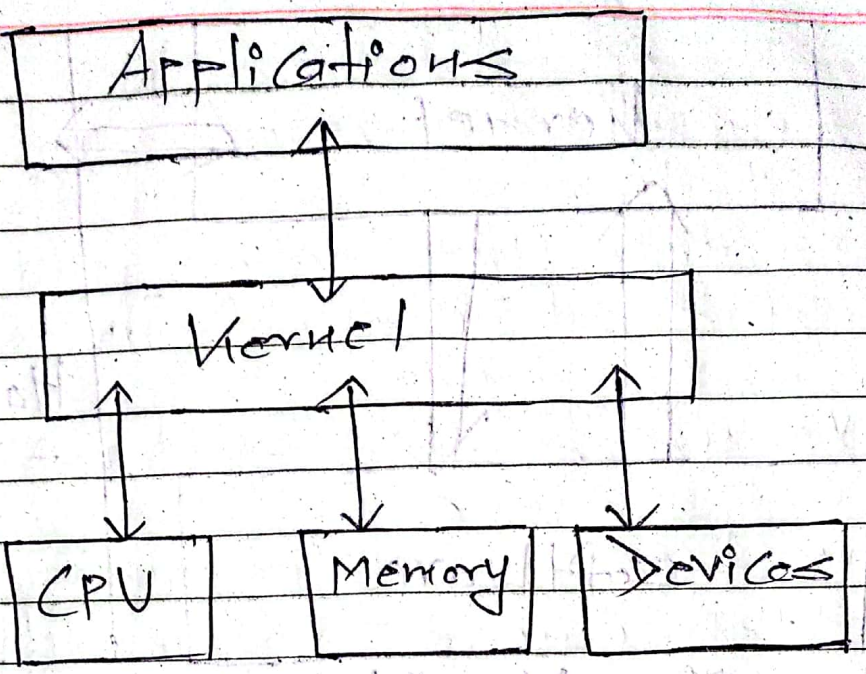


Fig: Architecture of kernel

Types of Kernel

1. Monolithic Kernel

-> It is one of types of kernel where all operating system services operate in kernel space.

-> It has dependencies between system components.

-> It has huge lines of code which is complex.

Example:
Unix, Linux, Open VMS, XTS-400 etc., BSD

→ The OS needs to be modified if there is addition of a new service

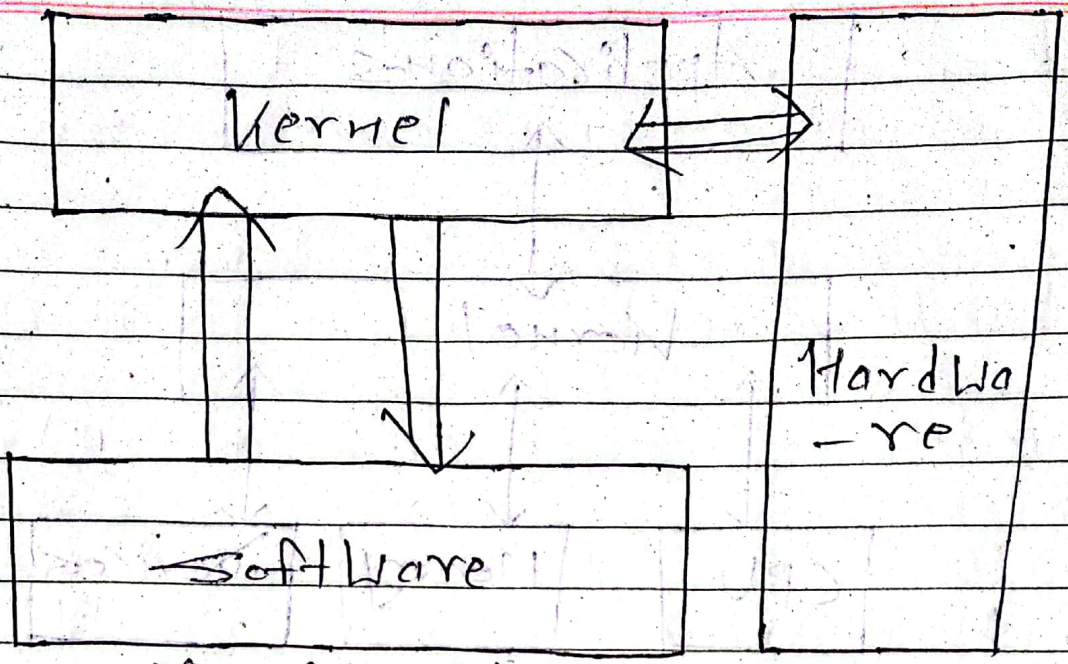


Fig: Monolithic Kernel

Advantages:
→ It has good performance.
→ Provide CPU scheduling, memory scheduling and file management only through system calls.

Disadvantages:
→ It has dependencies between system component and lines of code in millions.
→ Failure of a service leads to the failure of entire system

2. Micro Kernel

→ It is kernel types which has mini-malist approach.
→ It has virtual memory and thread scheduling.
→ It is more stable with less service in kernel space.
→ It puts rest in user space

Example:
Mach, QNX, Lu, Minix, KA2 etc

→ Communication is done through message passing and that reduces the speed of execution.

Advantages:
→ More Stable
→ New services can be easily added

Disadvantages:
→ Huge amounts of system calls and context switches
→ Communication reduces overall execution time

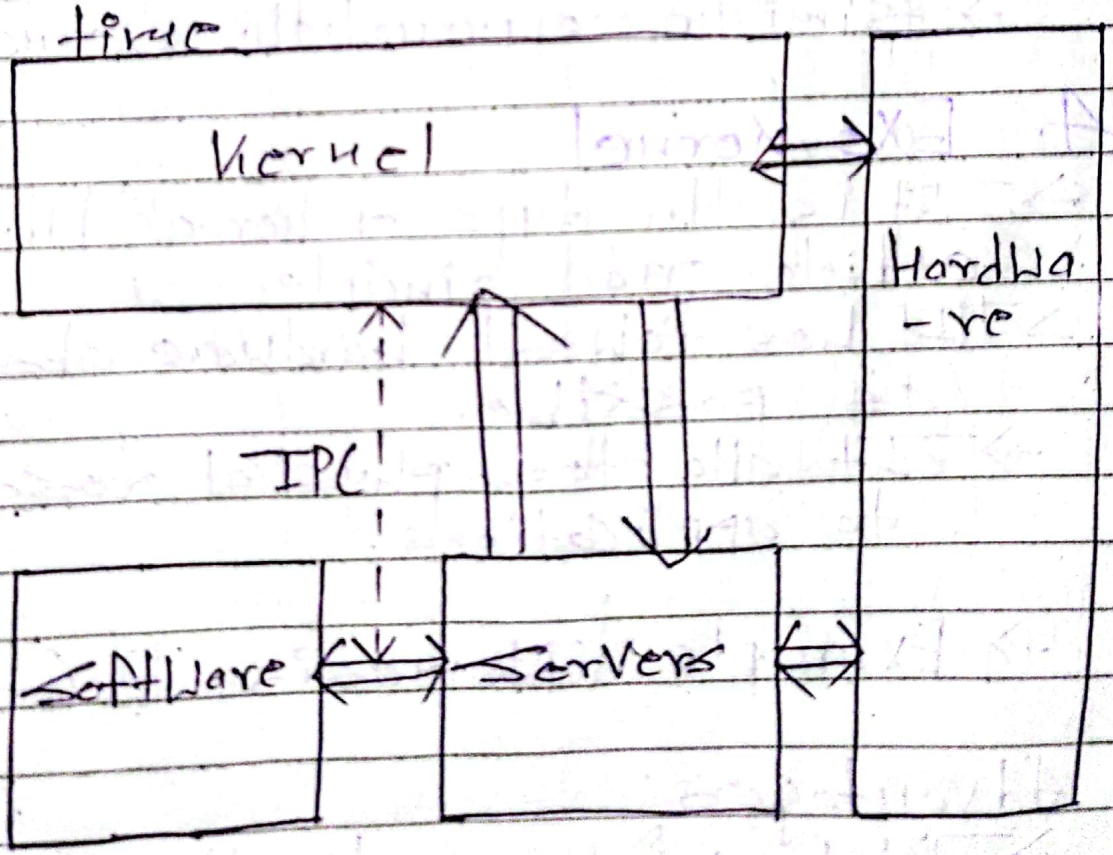


Fig: Microkernel

3. Hybrid Kernel

→ It is the combination of both monolithic kernel and microkernel.

→ It has speed and design of monolithic kernel and modularity and stability of microkernel.

Example: Windows NT, NetWare, BeOS etc

Advantages:

→ It combines both monolithic kernel and microkernel

Disadvantages:

→ similar to monolithic kernel

4. Exo Kernel

→ It is the type of kernel which follows end-to-end principle

→ It has fewest hardware abstractions as possible.

→ It allocates physical resources to applications.

→ Example: Nemesis, ExOS, etc

Advantages:

→ It has fewest hardware abstractions

Disadvantages:
→ There is more work for application developers.
→ Has a complex design

5. Nano Kernel:
→ This type offers hardware abstraction without system services.
→ Due to micro kernels also not having system services these two have become analogous to each other.
→ The code of the kernel is very small and the kernel supports nanosecond clock resolution.

Advantages:
→ offers hardware abstractions without system services

Disadvantages:
→ It is used less due to being similar to micro kernels

Example:
EROS, etc.

Context Switching (User mode and Kernel mode)

→ A Context switch is a procedure the a Computer's CPU follows to change from one task (or process) to another while ensuring that the tasks do not conflict.

→ Context switching involves storing the context or state of a process so that it can be reloaded when required and execution can be resumed from the same point as earlier.

→ This is a feature of a multitasking operating system and allows a single CPU to be shared by multiple processes.

→ Context switching can be described in the kernel performing the following activities with regard to processes on the CPU:

1. Suspending the progression of one process and storing the CPU's state for that process somewhere in memory.

2. Retrieving the context of the next process from memory and restoring in the CPU's registers and

3. Returning to the location indicated by the program counter in order to resume the process.

*) → There are three situations where a Context switch needs to occur (When does Context switching take place?). They are:

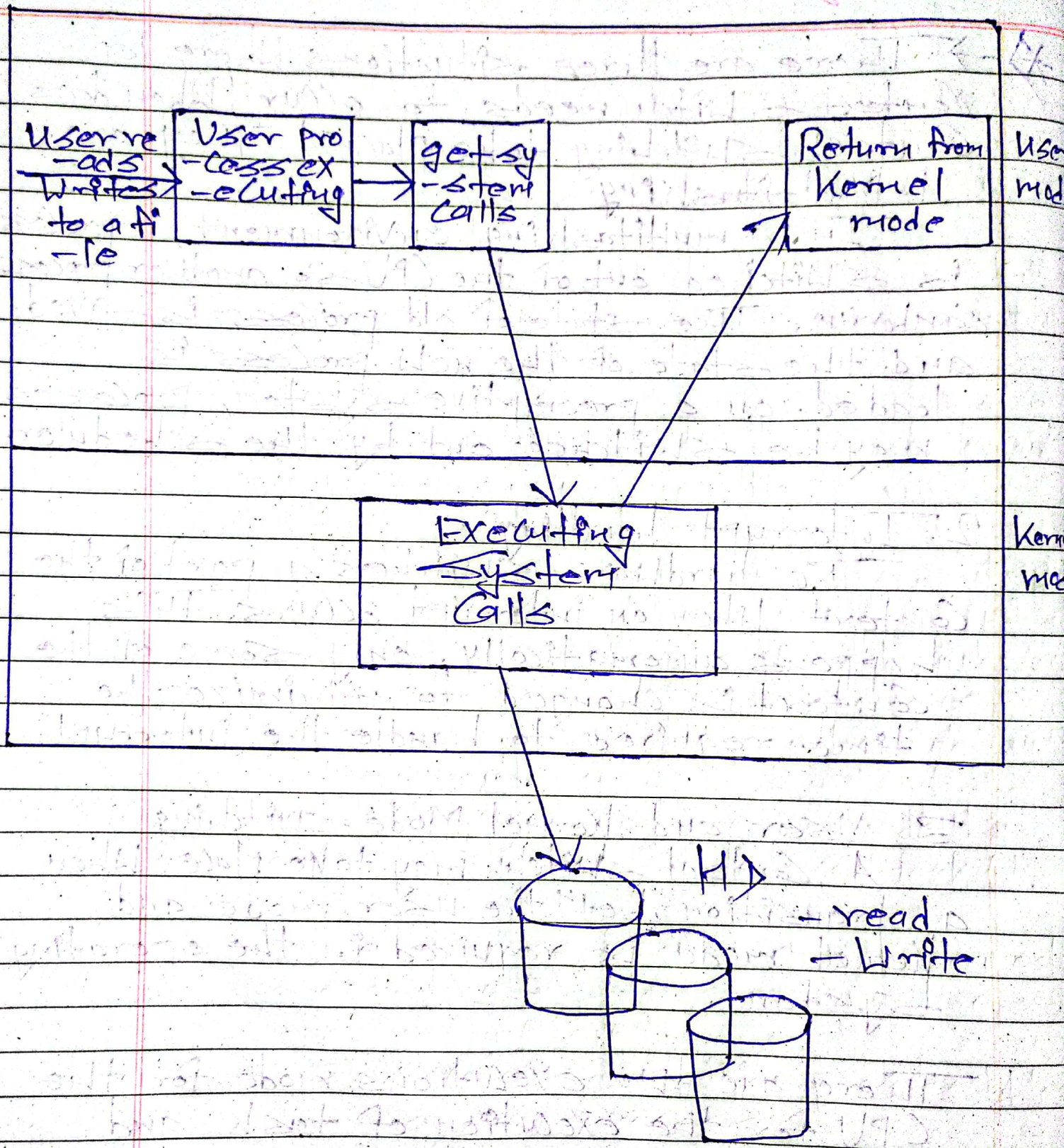
1. Multitasking:
   In a multitasking environment, a process is switched out of the CPU so another process can run. The state of old process is saved and the state of the new process is loaded. On a preemptive system, processes may be switched out by the scheduler.

2. Interrupt handling
   The hardware switches a part of the Context when an interrupt occurs. This happens automatically. only some of the Context is changed to minimize the time required to handle the interrupt.

3. User and Kernel Mode switching:
   A Context switch may take place when a transition bet^n the user mode and Kernel mode is required in the operating system.

There are two executions mode for the CPU for the execution of task and they are:

1. User mode and
2. Kernel mode


User re
-ads
Writes
to a fi
-le
User pro
-cess ex
-ecuting
get sy
-stem
calls
Return from
Kernel
mode
Executing
System
Calls
HD
-read
-write

1. Kernel Mode

- When CPU is in kernel mode, the code being executed can access any memory address and any hardware resource.
- It is a very privileged and powerful mode.
- If a program crashes in kernel mode, the entire system will be halted.
- In kernel mode, the executing code has complete and unrestricted access to the underlying hardware
- Core operating system components run in kernel mode.

2. User Mode

- When CPU in user mode, the program don't have access to memory and hardware resources.
- In user mode, if any program crashes, only that particular program is halted. That means the system will be in a safe state even if a program in user mode crashes.
- Most program in OS run in user mode
- Applications run in user mode.

Kernel Implementation of Process

→ Most of the operating system executes within the environment of a user process. We need two mode: User mode and Kernel mode.

→ Some portion of the operating system operates as a separate process known as Kernel process.

→ A Kernel process is a process that is created in the Kernel protection domain and always executes in the Kernel protection domain.

→ Kernel process can be used in subsyste, by complex device drivers and by system calls. They can also be used by interrupt handlers to perform asynchronous processing not available in the interrupt environment.

→ Process Creation in Unix is made by means of Kernel system call fork().

→ When a process issues a fork request, the operating system perform the following function:

i. It allocates a slot in process table for a new process.

ii. It assigns a unique process ID to a child process.

iii. It assigns the child process to a ready run state.

iv. It returns the ID number of the child process to parent process and Zero value to a child process.

→ All these work is accomplished in kernel mode in the parent process. When kernel has completed these functions, it can do one of the following routine

i. stay in parent process:
Control return to the user mode at the point of the fork call of the parent.

ii. Transfer Control to the child process:
The child process begins executing at the same point in the code as the parent, at the return from fork call.

iii. Transfer Control to other process:
Both the child and the parent process are left in ready to run state.

Example:

```
#include <stdio.h>
#include<sys/types.h>
#include <unistd.h>
int main()
```

5

```
// make two process which run same
// program after this instruction
fork();
printf("I am santosh!");
return 0;
}
```

output:

```
I am santosh!
I am santosh!
```

Number of times I am santosh! printed is equal to number of process created.

Total no. of processes = 2^n Where n is no. of fork system calls

So here $n=1$, $2^1 = 2$

Banker's Algorithm:

Resource-Request Algorithm for process Pi

Request = request vector for process Pi.
If Request_i[j] = k then process Pi wants k instances of resource type Rj.

1. If Request_i ≤ Need, go to step 2. otherwise, raise error condition, since process has exceeded its maximum claim.

2. If Request_i ≤ Available, go to step 3. otherwise Pi must wait, since resources are not available.

3. pretend to allocate requested resources to Pi by modifying state as follows:
   a. Available = Available − Request.
   b. Allocation_i = Allocation_i + Request_i.
   c. Need = Need_i − Request_i;

Safe = No deadlock occur = Safe sequence
Unsafe = Deadlock occur = no Safe sequence

Banker's Algorithm Numericals

Q.) Instance
Total A = 10, B = 5, C = 7
(Current)

Process	Allocation A	B	C	Max Need A	B	C	Available A	B	C	Remaining Need A	B	C
P_1	0	1	0	7	5	3	3	3	2	7	4	3
P_2	2	0	0	3	2	2	5	3	2	1	2	2
P_3	3	0	2	9	0	2				6	0	0
P_4	2	1	1	4	2	2	7	4	3	2	1	1
P_5	0	0	2	5	3	3	7	4	5	5	3	1
Total	7	2	5				7	5	5			
							10	5	7			

Available/Current Available = Total − Total Allocation

For A = 10 − 7 = 3
For B = 5 − 2 = 3
For C = 7 − 5 = 2

Remaining Need = Max Need − Allocation

Note:
If we can't fulfill the remaining need of any one process with Available then deadlock occur = unsafe

Safe sequence लेख्न दिया = उसको Remaining need पहिला fulfill हुन्छ त्यसलाई पहिला लेख्ने and so on.

Note:

*) If we can fulfill remaining need with available then its allocation also get free and availability increase.

New available = Allocation + previous available

Safe sequence:

P2, P4, P5, P1, P3

P2 → P4 → P5 → P1 → P3

Note:

Current Availability ≥ Remaining Need

एकचोटी Safe sequence मा आएको लाई दोस्रो चोटि नलेख्ने